

Calendar Whisperer

Book a Meeting from a Thread

Parse, check availability, book — one click.

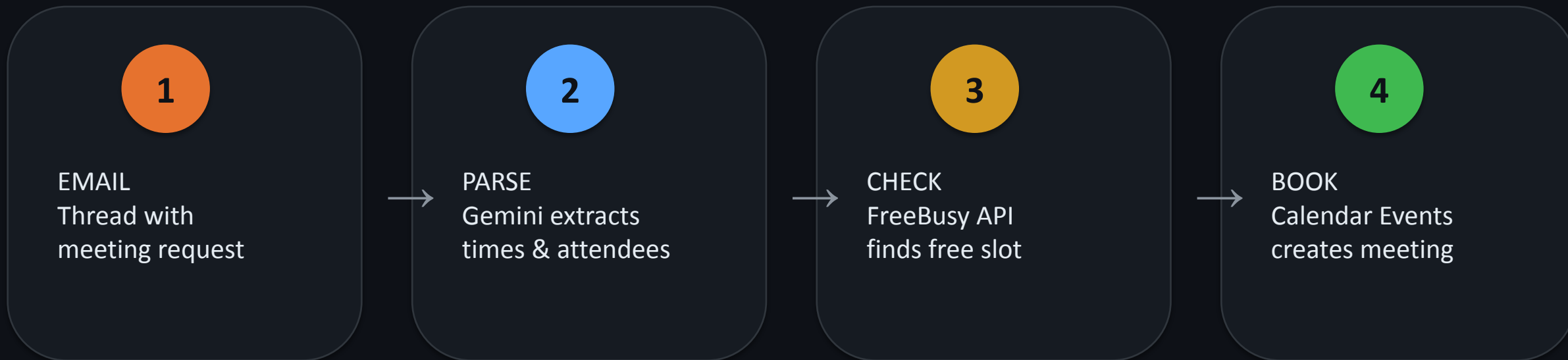
MS 3.2
Skill Sprint

Gemini + Calendar
API

FreeBusy
Events API

The Flow

Email thread → Parse → Check availability → Book event



If no slot is free → user is informed. No silent failures.

What We're Building Today

Four functions in a new `calendar_engine.py` + UI integration.

`parse_meeting_request()`

Gemini extracts times, attendees, topic, duration from thread

`check_availability()`

FreeBusy API checks if a proposed time slot is free

`find_free_slot()`

Iterates proposed times, returns first available slot

`create_event()`

Calendar Events API creates the meeting with attendees

Google Calendar API Setup

Same Cloud project, same credentials.json — just add a scope.

1. console.cloud.google.com → APIs & Services → Enable APIs

2. Search "Google Calendar API" → Click Enable

3. Add scope to engine.py: "https://...auth/calendar"

4. Delete token.json → re-authenticate (one time)

5. Fresh token now has: gmail.readonly + gmail.send + calendar

One token file, three scopes. Clean.

parse_meeting_request()

Gemini reads the thread and extracts structured meeting details.

INPUT: Email thread

"Can we meet Tuesday at 2pm,
Wednesday at 10am, or
Thursday at 3pm?
45-minute call to discuss
the Q3 roadmap."



OUTPUT: Structured JSON

```
{  
  proposed_times: [3 ISO dates],  
  attendees: [emails],  
  topic: "Q3 Roadmap Review",  
  duration_minutes: 45  
}
```

Gemini 2.5 Flash — fast, cheap, great at structured extraction.

FreeBusy API — check_availability()

"Is this slot actually free?" — ask Google Calendar.

```
service.freebusy().query(body={  
    "timeMin": "2026-06-24T14:00:00Z",  
    "timeMax": "2026-06-24T14:45:00Z",  
    "items": [{"id": "primary"}]  
}).execute()
```

Response:

```
{ "calendars": { "primary": {  
    "busy": [] ← empty = FREE!  
    }  
}}
```

find_free_slot() iterates proposed times → returns first with busy: []

create_event() — Booking the Meeting

Calendar Events API creates the event and sends invitations.

```
event = {  
  "summary": "Design Review Sync",  
  "start": {"dateTime": "2026-06-24T14:00:00Z", "timeZone": "UTC"},  
  "end": {"dateTime": "2026-06-24T14:45:00Z", "timeZone": "UTC"},  
  "attendees": [{"email": "sofia@designlabs.io"}]  
}
```

```
service.events().insert(  
  calendarId="primary", body=event, sendUpdates="all"  
).execute()
```

sendUpdates="all" → attendees get a Google Calendar invitation email.

Live Demo — Book Meeting Button

Meeting-request thread → Approve → Book Meeting → Event on Calendar

App flow:

1. Pull & Triage → meeting-request detected
2. Generate Draft → reply confirming time
3. Approve Draft
4. Click "Book Meeting" button
5. Event appears on Google Calendar

Result:

✓ Meeting booked!

Design Review Sync

Tue Jun 24, 2:00 PM - 2:45 PM

Attendees: sofia@designlabs.io

[\[View in Google Calendar\]](#)

Two buttons: Send (reply email) + Book Meeting (calendar event). Both in one UI.

What You Did Today



Enabled Google Calendar API + added calendar scope



Built `parse_meeting_request()` — Gemini extracts meeting details



Built `check_availability()` + `find_free_slot()` — FreeBusy API



Built `create_event()` — books the meeting on your calendar



Added Book Meeting button to app.py's Approval Gate

Tomorrow — MS 3.3

The Chain

Chain multiple skills
into one pipeline.

Read → Triage → Draft
→ Send → Book

All automated with
configurable human
checkpoints.